

NALOPAY PAYMENT & HOSTED CHECKOUT API DOCUMENTATION

Unified documentation for both NALOPAY Core API and NALOPAY Checkout API.

NALOPAY PAYMENT API V1

Overview

NALOPAY is a secure and reliable payment API designed to facilitate seamless transactions between merchants and customers. It enables businesses to accept, process, and manage payments effortlessly.

This documentation provides comprehensive details on authentication, endpoint usage, error handling, and sample requests to help you test and integrate the API with ease.

Prerequisites

To successfully integrate with the NALOPAY API, ensure you have the following details:

REQUIREMENT	DESCRIPTION
Merchant ID	A unique identifier assigned to your merchant account.
Basic Auth token	Authentication credentials provided for your account. Must be included in the Authorization header of each request.
Secret Key	A confidential key for signing requests and generating hashes.

Endpoints

1. Generate Token

- Method: POST
- URL: `{{baseUrl}}/clientapi/generate-payment-token/`
- Description: The first step in the collection process is to generate a payment token. This endpoint creates a **JWT token** for the specified merchant, which will be used in subsequent collection requests. It requires a valid `merchant_id` in the request body and a valid **Basic Auth token** in the request header.

Request Parameters

PARAMETER	DESCRIPTION
merchant_id	Your unique merchant ID

Headers

HEADER	DESCRIPTION
Authorization	Basic Auth token (Base64 encoded)
Content-Type	application/json

Sample Request Body

```
json

{
  "merchant_id": "1234567890"
}
```

Sample Response

```
json

{
  "success": true,
  "code": "TOKEN-CRTD-0050",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtZXJjaGFudF9pZCI6Im1ucUREaG1Tc3I2WThEUHk4"
  }
}
```

2. Create Collection

- Method: POST
- URL: `{{baseUrl}}/clientapi/collection/`
- Description: Creates a new payment collection for a merchant.

Headers

HEADER	DESCRIPTION
Content-Type	application/json
token	Token returned from Generate Token endpoint (JWT)

Request Fields

Field	Type	Required	Description
<code>merchant_id</code>	string	Yes	Your unique merchant identifier
<code>service_name</code>	string	Yes	"MOMO_TRANSACTION" for mobile money
<code>trans_hash</code>	string	Yes	HMAC SHA256 hash for verification
<code>account_number</code>	string	Yes	Customer's mobile money number
<code>account_name</code>	string	Yes	Customer's name
<code>network</code>	string	Yes	"MTN", "AT", or "TELECEL"
<code>amount</code>	number/string	Yes	Payment amount (min: 0.01)
<code>reference</code>	string	Yes	Your unique transaction reference
<code>callback</code>	string	Yes	Your webhook URL for payment updates
<code>description</code>	string	No	Transaction description

How to generate `trans_hash`

Construct a message by concatenating the following fields in order, without separators, then compute an HMAC-SHA256 of the message using your merchant secret key.

Order of fields:

- `merchant_id`
- `account_number`
- `amount`
- `reference`

Example message (no separators):

Plain Text

MERCH123024123456750.00TXN-123

Compute: `HMAC_SHA256(message, merchant_secret_key)` and send the resulting hex string as `trans_hash`.

Sample Request Body

```
{
  "merchant_id": "123456789",
  "service_name": "MOMO_TRANSACTION",
  "trans_hash": "0000000000000000000000000000000000000000000000000000000000000000",
  "account_number": "1234567890123456789012345678901234567890123456789012345678901234",
  "account_name": "John Doe",
  "network": "MTN",
  "amount": "10000000000000000000000000000000000000000000000000000000000000000",
  "reference": "1234567890123456789012345678901234567890123456789012345678901234",
  "callback": "https://example.com/webhook",
  "description": "Mobile money transaction"
}
```

json

```
{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP",
  "service_name": "MOMO_TRANSACTION",
  "trans_hash": "b004395b29a61c3df913857032b40ad0e19c3ed1b9b853d85ae2c03cc8f3e0ba",
  "account_number": "233597990630",
  "account_name": "Abdul Razak",
  "description": "description",
  "reference": "REF_2024_001",
  "callback": "http://localhost:8033",
```

Sample Response

json

```
{
  "success": true,
  "code": "PAY-CRTD-0055",
  "data": {
    "order_id": "FPewDB25nodznJawcNykhx",
    "status": "PENDING",
    "amount": 50,
    "timestamp": "2025-08-18 12:41:23",
    "otp_code": "None*252#"
  }
}
```

3. Callback (Webhooks)

When a collection transaction is processed (successful or failed), the payment gateway sends a POST request to your callback URL.

- Method: POST
- URL:
- Headers:

Plain Text

Content-Type: application/json

Callback Payloads

- Successful Payment

json

```
{
  "order_id": "LVFf4MHD2xJ7yJ7uW9ZyMi",
  "status": "COMPLETED"
```

```
    "status": "COMPLETED",
    "amount": "50.00",
    "charges": "0.00",

    "transaction_fee": "0.00"
}
```

- Failed Payment

json

```
{
  "order_id": "h7bTLkH4ZZd2m5kgd8XvGf",
  "status": "FAILED",
  "amount": "100.00",
  "charges": "0.00",
  "transaction_fee": "0.00"
}
```

- With Extra Data (Optional)

json

```
{
  "order_id": "oEgJmCznUUa89hWKrAjfCr",
  "status": "COMPLETED",
  "amount": "100.00",
  "charges": "0.00",
  "transaction_fee": "0.00",
  "extra_data": {
    "key": "value",
    "custom_field": "custom_value"
  }
}
```

4. Check Status

- Method: POST
- URL: `{{BASE_URL}}/clientapi/collection-status/`
- Description: Use this endpoint to check the status of a payment collection by providing the `merchant_id` and `order_id`. The response will indicate the current status and amount of the specified collection order.

Request Parameters

PARAMETER	DESCRIPTION
merchant_id	Your unique merchant ID
order_id	The order ID returned when the collection was created

Headers

HEADER	DESCRIPTION
Content-Type	application/json

Sample Request Body

```
json

{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP",
  "order_id": "2jXVMbTidVjSvMGsD2fNWR"
}
```

Sample Response (Success)

```
json

{
  "success": true,
  "code": "PAY-STAT-0080",
  "data": {
    "status": "PENDING",
    "amount": 0.11
  }
}
```

API SAMPLE EXAMPLES

POST Generate Token

{{baseUrl}}clientapi/generate-payment-token/

HEADERS

Authorization Basic
8d29d845b9e4cab596d6bed125cb10989266f36c9ec6cdbe0741b8b99c2dcca
5d59d6254f9446630cb43f8bd1d9f332bf59fe285a86742955becfbd9cce4a2d
c

Body raw (json)

json

```
{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP"
}
```

POST Create collection

{{baseUrl}}/clientapi/collection/

HEADERS

token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtZXJjaGFudF9pZCI6Im1ucUREaGtC3I2WThEUHk4RDJYb0QiLCJiYXNpY19hdXRoX2hYWRlciI6Ikc2IjIjGFjMWMzYWQxNWUxZGZhNzVjOWFIZmY2YzQ3MWNmMjc3MjQwOWRjYzE5NTRIODFiZjIwM2YzYjAzNjk3NDNjNzYwZmE1ODk5N2NmNDBmODI1ZDc1YWVmYzJhMzJIZDlkYzYzNGE4YWRIN2NhOTBIZTU2ZDdiY2lwZGY1MjZiMjViliwiaWF0IjoxNzU1NTlwMTA1LCJleHAiOiE3NTU1MjEwMDV9.pRARV0UXCAfEaZigbuHRnleFGWf6YDuBbJ2HUj840o0

Body raw (json)

json

```
{
  "merchant_id": "mnqDDhiSsr6Y8DPy8D2XoD",
  "service_name": "MOMO_TRANSACTION",
  "trans_hash": "69cba164a6b28202ffe9294c7977586309699c22c526f9a96ecb0f15500be7ae",
  "account_number": "0597990630",
  "account_name": "Alexander Alex",
  "description": "description",
  "reference": "REF_2024_001",
  "network": "MTN",
  "amount": 0.2
}
```

POST Check status

{{BASE_URL}}/clientapi/collection-status/

Body raw (json)

json

```
{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP",
  "order_id": "2jXVMbTidVjSvMGsD2fNWR"
}
```

NALOPAY HOSTED CHECKOUT API V1

Overview

NALOPAY Hosted Checkout API is the quickest way for you to integrate NALOPAY wherever you accept payments. A simple API call allows an application to get a URL to a NALOPAY-hosted checkout page for a buyer to pay for goods and services using either **Mobile Money (MoMo)** or **Card**.

How it works

The NALOPAY Checkout API works as follows:

1. When a buyer wants to make a purchase, your application uses the Checkout API to create a NALOPAY-hosted checkout page.
2. The API then returns the URL to the checkout page.
3. Your application can then provide the checkout page URL to the buyer.
4. After the buyer opens the checkout page and pays for the purchase, NALOPAY redirects the buyer back to the referral page.

Prerequisites

Ensure you have the following to integrate with the Hosted Checkout API:

REQUIREMENT	DESCRIPTION
Merchant ID	A unique identifier assigned to your merchant account.
Basic Auth token	Authentication credentials provided for your account. Must be included in the Authorization header when generating a token.
Secret Key	A confidential key for signing requests and generating hashes.

Endpoints

1. Generate Token

- Method: POST
- URL: `{{BASE_URL}}/clientapi/generate-payment-token/`
- Description: The first step in the checkout process is to generate a payment token. This endpoint creates a JWT token for the specified merchant, which will be used in subsequent checkout requests. It requires a valid `merchant_id` in the request body and a valid Basic Auth token in the request header.

Request Parameters:

Parameter	Description
<code>merchant_id</code>	Your unique merchant ID

Headers:

Header	Description
Authorization	Basic Auth token (Base64 encoded)
Content-Type	application/json

Sample Request Body:

```
json
{
  "merchant_id": "1234567890"
}
```

Sample Response:

```
json
```

```
{
  "success": true,
  "code": "TOKEN-CRTD-0050",
  "data": {
    "token": "<JWT token>"
  }
}
```

2. Create Checkout Session

- Method: POST
- URL: `{{BASE_URL}}/checkout/session/`
- Description: Creates a hosted checkout session for the merchant and returns a redirect URL that the shopper is sent to for payment.

Request Parameters

Headers

HEADER	DESCRIPTION
Content-Type	application/json
token	Token returned from Generate Token endpoint (JWT)

The request body must contain two top-level keys:

- `merchant` : An object containing merchant and order details.
- `summary` : An object containing product and order summary details.

`merchant` Object Fields:

Field	Description
<code>merchant_id</code>	Your unique merchant ID
<code>order_id</code>	Unique identifier for the order
<code>customer_name</code>	Name of the customer
<code>referral_url</code>	URL where the customer was referred from (e.g., your shop's landing page)
<code>callback_url</code>	Webhook URL to receive payment updates
<code>trans_hash</code>	HMAC SHA256 hash for verification
<code>reference</code>	Unique reference for the order/transaction
<code>mode</code>	Payment mode (e.g., MOMO, or CARD default is ANY)

`summary` Object Fields:

Field	Description
products	Array of product line items (name, count, price)
item_count	Total number of items in the order
total_price	Total price for the order

Optional Fields:

Field	Description
metadata	Optional custom key-value data that can be associated with a product for tracking or reference purposes.

How to Generate `trans_hash`

Construct a message by concatenating the following fields in order, without separators, then compute an HMAC-SHA256 of the message using your merchant secret key.

Order of fields:

- merchant_id
- order_id
- total_price
- reference

Example message (no separators):

Plain Text

MERCH123024123456750.00TXN-123

Compute: `HMAC_SHA256(message, merchant_secret_key)` and send the resulting hex string as `trans_hash`.

Sample Request Body

```

json

{
  "merchant": {
    "merchant_id": "mnqDDhiSsr6Y8DPy8D2XoD",
    "order_id": "ORDER12345",
    "customer_name": "GADDAFI",
    "referral_url": "https://yourshop.com",
    "callback_url": "https://yourshop.com/payment-callback",
    "trans_hash": "7b1a55afaf4dc7a5f45376934084f4f834e9733daab0e59817f2bf60cc625075",
    "signature": "855f88818881"
  }
}

```

```
    "reference": "REF_2024_001",
    "mode": "MOMO"
  },
}
```

Sample Success Response

```
json

{
  "success": true,
  "code": "CHECKOUT-CRTD-0071",
  "data": {
    "checkout_url": "{{CheckoutBaseUrl}}?id=84887d1d-b783-49ea-a528-13332e274667",
    "checkout_timeout": 1800
  }
}
```

3. Callback (Webhook)

When a checkout payment is processed (successful or failed), the gateway sends a POST request to your callback URL.

Successful Payment Callback

Endpoint: POST

Headers:

Plain Text

Content-Type: application/json

Callback Payload (Successful Payment):

```
json

{
  "order_id": "FPewDB25nodznJawcNykhx",
  "status": "COMPLETED",
  "amount": "50.00",
  "charges": "0.50",
  "transaction_fee": "0.00",
  "extra_data": {
    "products": [
      {
        "name": "Premium Subscription",
        "count": 1,

```

Callback Payload (Successful Card Payment):

```
json

{

```

```
"order_id": "FPewDB25nodznJawcNykhx",
"status": "COMPLETED",
"amount": "50.00",

"charges": "0.00",
"transaction_fee": "1.00",
"email": "john.doe@example.com",
"extra_data": {
  "products": [
    {
```

Failed Payment Callback

Callback Payload (Failed Payment):

json

```
{
  "order_id": "FPewDB25nodznJawcNykhx",
  "status": "FAILED",
  "amount": "50.00",
  "charges": "0.00",
  "transaction_fee": "0.00",
  "extra_data": {
    "products": [
      {
        "name": "Premium Subscription",
        "count": 1,
```

4. Check Status

- Method: POST
- URL: `{{BASE_URL}}/clientapi/collection-status/`
- Description: Use this endpoint to check the status of a payment collection by providing the `merchant_id` and `order_id`. The response will indicate the current status and amount of the specified collection order.

Request Parameters

PARAMETER	DESCRIPTION
merchant_id	Your unique merchant ID
order_id	The order ID returned when the collection was created

Headers

HEADER	DESCRIPTION
Content-Type	application/json

Sample Request Body

json

```
{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP",
  "order_id": "2jXVMbTidVjSvMGsD2fNWR"
}
```

Sample Response (Success)

```
json

{
  "success": true,
  "code": "PAY-STAT-0080",
  "data": {
    "status": "PENDING",
    "amount": 0.11
  }
}
```

API SAMPLE EXAMPLES

POST Generate Token

{{baseUrl}}/clientapi/generate-payment-token/

HEADERS

Authorization

Basic

8d29d845b9e4cab596d6bed125cb10989266f36c9ec6cdbe0741b8b99c2dcca
5d59d6254f9446630cb43f8bd1d9f332bf59fe285a86742955becfbd9cce4a2d
c

Body raw (json)

```
json

{
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP"
}
```

POST Create checkout session

{{baseUrl}}/checkout/session/

HEADERS

token {{TOKEN}}

Body raw (json)

json

```
{
  "merchant": {
    "merchant_id": "mnqDDhiSsr6Y8DPy8D2XoD",
    "order_id": "ORDER12345",
    "customer_name": "GADDAFI",
    "referral_url": "https://yourshop.com",
    "callback_url": "https://yourshop.com/payment-callback",
    "trans_hash": "0cb9c7dccb4bd1588f919f61779fc1491af73e34dd94b1c695403da4ebacbe31",
    "reference": "REF_2024_001",
    "mode": "MOMO"
  },
  "summary": {
    "products": [
      {
        "name": "Wireless Mouse",
        "count": 2,
        "price": "25.00"
      },
      {
        "name": "Mechanical Keyboard",
        "count": 1,
        "price": "120.00"
      }
    ],
    "item_count": 3,
    "total_price": "170.00"
  }
}
```

POST Create checkout session (with metadata)

{{baseUrl}}/checkout/session/

HEADERS

token {{TOKEN}}

Body raw (json)

json

```
{
  "merchant": {
    "merchant_id": "mnqDDhiSsr6Y8DPy8D2XoD",
    "order_id": "ORDER12345",
    "customer_name": "GADDAFI",
    "referral_url": "https://yourshop.com",
    "callback_url": "https://yourshop.com/payment-callback",
    "trans_hash": "7b1a55afaf4dc7a5f45376934084f4f834e9733daab0e59817f2bf60cc625075",
    "reference": "REF_2024_001",
    "mode": "MOMO"
  },
  "summary": {
    "products": [
      {
        "name": "Wireless Mouse",
        "count": 2,
        "price": "25.00",
        "metadata": {
          "sku": "WM-AX12",
          "color": "Black",
          "warranty_months": 12
        }
      },
      {
        "name": "Mechanical Keyboard",
        "count": 1,
        "price": "120.00",
        "metadata": {
          "sku": "MK-BL88",
          "switch_type": "Blue",
          "warranty_months": 24
        }
      }
    ],
    "item_count": 3,
    "total_price": "170.00"
  }
}
```

POST Check status

{{BASE_URL}}/clientapi/collection-status/

Body raw (json)

json

```
{  
  "merchant_id": "hCuK9z9yoYMZ8yvtH7LUHP",  
  "order_id": "2jXVMbTidVjSvMGsD2fNWR"  
}
```